

Week 8 - ML

📅 Start Date	@22 December 2025
☰ Weeks	Week 8

Sentiment Analysis

- Subfield of **Natural Language Processing (NLP)** → Sentiment Analysis → Use machine learning algorithm to classify documents based on sentiment
- Test with IMDb

IMDb Movie Review Data For Text Processing

- Sentiment analysis → sometimes called **Opinion Mining**
- Popular task → Classification of documents based on expressed opinion or emotion of author

Bag-of-Words Model

- **Bag-of-Words model** → Allows us to present text as numerical feature vectors
- **Summarised:**
 1. Create vocabulary of unique tokens - e.g. words from entire set of documents
 2. Construct feature vector from each document → The count of how often each words occur
- Since unique words in each document only represent small subset → feature vector will mainly consist of 0 → call them **sparse**

Words To Feature Vectors

- **Raw Term Frequencies** - $tf(t,d)$ - number of times term t , occurs in document, d → **fundamental metric in NLP**

- In Bag-of-Words model → word/term order in sentence/document doesn't matter

N-gram Model

- Sequence of items in Bag-of-Words → **called 1-gram or unigram model** → each token in vocab represents a single word
- Contiguous sequence of items in NLP → **called n-grams**
- Choice of number, n depends on application
- E.g:
 - 1-gram: "the", "sun", "is", "shining"
 - 2-gram: "the sin", "sum is", "is shining"
- n can be switched by `ngram_range` in `CountVectorizer`

Assessing Word Relevancy via Term Frequency-Inverse Document Frequency

- **Term Frequency-in-Verse Document Frequency (Tf-idf)** → Use to down-weight frequently occurring words in feature vectors
 - Product of term frequency + inverse document frequency

$$tf - idf(t, d) = tf(t, d) \times idf(t, d)$$

- $tf(t, d)$ → Term frequency
- $idf(t, d)$ → Inverse document frequency

$$idf(t, d) = \log \frac{n_d}{1 + df(d, t)}$$

- n_d = Total number of documents
- $df(d, t)$ = Number of documents, d , contain the term, t
- 1 = Optional → Purpose is to assigning a non-zero value to terms that occur in none of the training examples
- $\log$ = Ensured for low document frequencies are not given too much weight

- Scikit-Learn → `TfidfTransformer` → inverse document frequency:

$$idf(t, d) = \log \frac{1 + n_d}{1 + df(d, t)}$$

- $tf - idf$ → Scikit-Learn → $tf - idf(t, d) = tf(t, d) \times (idf(t, d) + 1)$
- Normally, you would have to normalise raw term frequencies before calculating $tf - idf$ → `TfidfTransformer` Normalises $tf - idf$ s.
 - Default also is `norm 'l2'`
- Returns vector length 1 by unnormalised vector, v / L2-norm:

$$v_{norm} = \frac{v}{||v||_2} = \frac{v}{\sqrt{v_1^2 + v_2^2 + \dots + v_n^2}} = \frac{v}{(\sum_{i=1}^n v_i^2)^{\frac{1}{2}}}$$

Cleaning Text Data

- First important step → Clean text data by stripping off all unwanted characters
- Accomplish this with REGEX → Regular expression
- Removing HTML markup → REGEX acceptable → won't use HTML markup further

Processing Documents Into Tokens

- Tokenise documents → Split individual words → Split cleaned document after whitespace characters
- Another technique
 - Word Stemming** → Transforming words to their root form → Allows us to map related words to the same stem
 - Original algorithm by Martin F. Porter (1979) → Known as **Porter Stemmer Algorithm**

Stemming Algorithms

- Porter Stemming Algorithm → Oldest and simplest stemming algorithm
- Other examples:**
 - Snowball Stemmer (Porter 2 or English Stemmer)

- Lancaster Stemmer (Paice/Husk Stemmer) → Aggressive compared to Porter → **Produces shorter and more obscure words**



Both faster than original Porter Stemmer

- Stemming can create non-real (e.g. 'thu' instead of 'thus')
 - **Lemmatization** → Technique to obtain canonical (grammatically correct) forms for individual words (lemmas)
 - Computationally more expensive and difficult → **Not worth the time and effort because it has little to no effect on text classification**
- **Stop Word Removal** → Extremely common words in all sorts of text that bear little to no useful information for distinguished different classes
 - Can
- **Naïve Bayes Classifier** → Very popular text classification classifier → **Easy to implement, computationally efficient, performs well on small datasets.**
 - Gained popularity through email spam filters

Working With Bigger Data - Online Algorithm + Out-of-Core Learning

- Computationally expensive to construct feature vector for large dataset



Uncommon to work with large datasets that exceed computer's memory

- **Out-of-Core Learning** → Allows us to work with large datasets → Fit classifier incrementally on smaller batches
- Later → we will use neural networks with out-of-core learning and stochastic gradient descent
- Out-of-Core learning is memory efficient

Word2Vec model

- Modern alternative to bag-of-words model

- Released by google in 2013
- Unsupervised learning algorithm → based on neural networks → Attempt to automatically learn relationship between words
- Idea, puts similar words in small clusters - via clever vector spacing

Topic Modelling With Latent Dirichlet Allocation

- Topic Modelling → Broad task of assigning topics to unlabelled text documents
 - Typical example → Categorisation of documents in large corpus of newspaper articles
 - Can be classified as clustering task (unsupervised learning)



Popular technique - Latent Dirichlet Allocation (LDA) → Don't confuse with Linear Discriminant Analysis (LDA)

Decomposing Text Documents With LDA

- LDA → Probabilistic model → Tries to find groups of words that appear together frequently across different documents
- Input to LDA is Bag-of-Words model
 - Splits matrix input to two near matrices:
 1. Document-to-Topic matrix
 2. Word-to-Topic matrix



LDA decomposes Bag-of-Words model in a way that if we multiply the two new matrices we get the Bag-of-Words input matrix

- Only downside → We must define number of topics beforehand → LDA Hyperparameter

Expectation-Maximization

- Scikit-Learn LDA uses Expectation-Maximization (EM)